

Project Presentation: Approximate Parallel Prefix Adders (AxPPA)

Name	Roll Number	Email
Bharat Kumar	IMT2023504	bharat.kumar@iiitb.ac.in
Vinay P Ramesh	IMT2023541	vinay.ramesh@iiitb.ac.in
R Vaikunth	IMT2023566	r.vaikunth@iiitb.ac.in
Surya Sumeet Singh	IMT2023590	sumeetsurya.singh@iiitb.ac.in
Pravin Kumar V	IMT2023595	pravinkumar.v@iiitb.ac.in
Shudharshan A	IMT2023602	shudharshan.a@iiitb.ac.in
Harshit Krishna R	IMT2023613	Harshit.krishna@iiitb.ac.in

Introduction

- ▶ Digital systems increasingly rely on low-power, high-speed arithmetic.
- ▶ Many applications (image processing, ML, DSP) tolerate small errors.
- ▶ Approximate computing improves:
 - ▶ • Area
 - ▶ • Speed
 - ▶ • Power
- ▶ Goal: Make adders faster and smaller with minimal accuracy loss.
- ▶ We must get the best trade-off between efficiency and accuracy to suit various applications.

Why Approximate Computing?

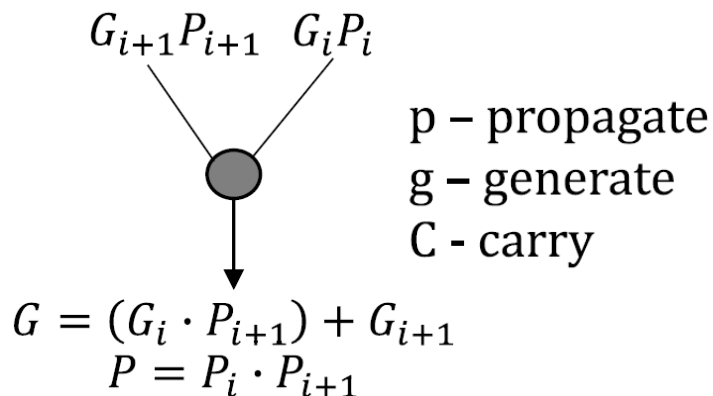
- ▶ Error-tolerant applications:
 - ▶ Graphics
 - ▶ Video processing
 - ▶ Neural Networks
 - ▶ Sensor data
- ▶ These domains don't require exact arithmetic.
- ▶ Bottleneck: Adders consume large area and delay.
- ▶ Approximation = performance boost. Lesser area and delay improves performance.

What Are Parallel Prefix Adders (PPA)?

- ▶ High-speed adders using prefix computation.
- ▶ Aim: Reduce carry propagation delay.
- ▶ Use Generate (G) and Propagate (P) logic.
- ▶ Types of AxPPAs implemented:
 - ▶ • Kogge-Stone
 - ▶ • Brent-Kung
 - ▶ • Sklansky
 - ▶ • Ladner-Fischer

Prefix Operators (Generate & Propagate)

- ▶ Propagate: $P(i) = A(i) \text{ XOR } B(i)$
- ▶ Generate: $G(i) = A(i) \text{ AND } B(i)$
- ▶ Prefix operation:
 - ▶ newP = prevP and prevP_left
 - ▶ newG = prevG or (prevP and prevG_left)
- ▶ Used to compute carry in logarithmic stages.



Algorithm 1 Pseudo Code for W-Bit PPA [56]

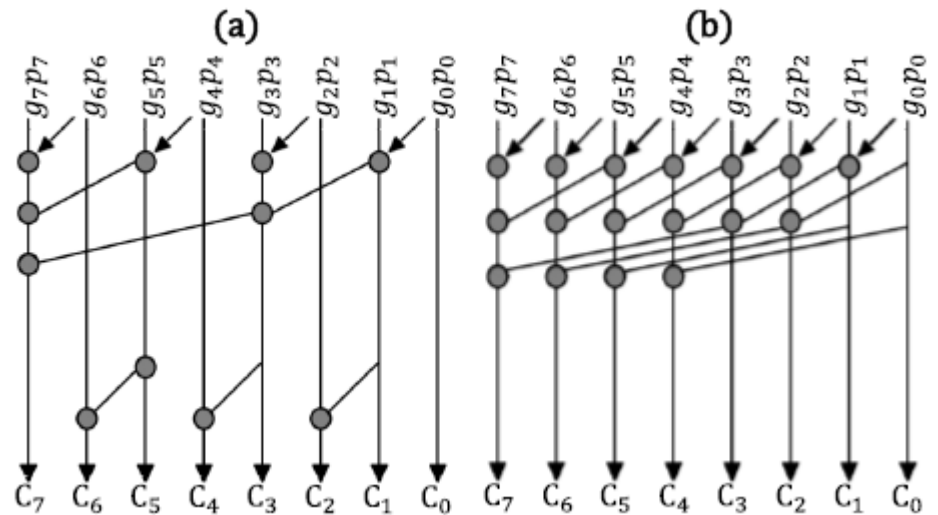
```

1: Input:  $A, B, cin, W$ 
2: Output:  $S$ 
3: for  $i = 1$  to  $W$  do
4:    $p(1, i) \leftarrow A(i) \text{ XOR } B(i)$ 
5:    $g(1, i) \leftarrow A(i) \text{ AND } B(i)$ 
6: end for
7:  $y \leftarrow 1$ 
8: for  $i = 2$  to (Delay calculation in Table II) do
9:   for  $j = 2 + y$  to  $W + 1$  do
10:     $P(i, j) \leftarrow p(i - 1, j) \text{ AND } p(i - 1, j - y)$ 
11:     $G(i, j) \leftarrow p(i - 1, j) \text{ AND } (g(i - 1, j - y) \text{ OR } g(i - 1, j))$ 
12:   end for
13:    $y \leftarrow 2^{j-1}$ 
14: end for
15:  $C(0) \leftarrow cin$ 
16: for  $i = 1$  to  $W + 1$  do
17:    $C(i) \leftarrow cin \text{ AND } P(W + 1, i) \text{ OR } G(W + 1, i)$ 
18:    $S(i) \leftarrow C(i - 1) \text{ XOR } p(1, i)$ 
19: end for
    
```

Implemented PPA Architectures

a) Brent-Kung:

- Balanced delay
- Least LUTs



b) Kogge-Stone:

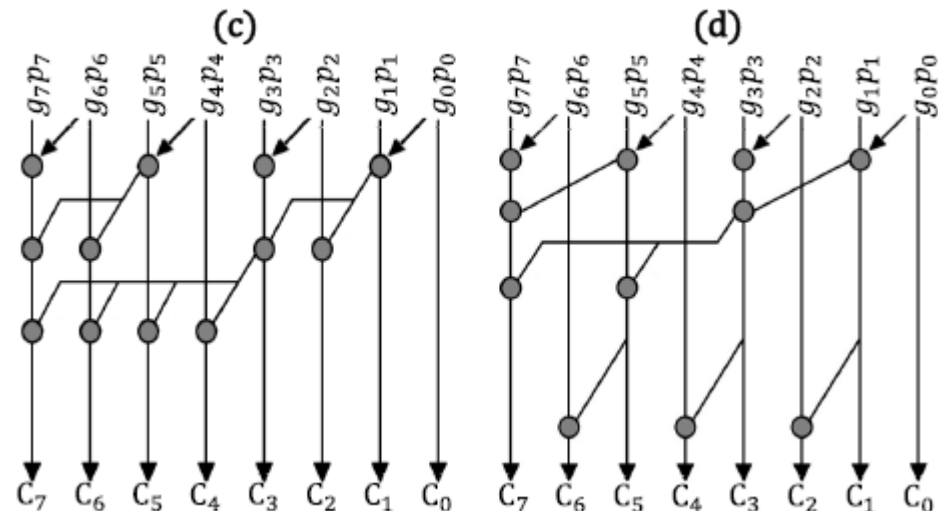
- Minimum delay
- Highest area

c) Sklanskey:

- High Fanout

d) Ladner-Fischer:

- Highest LUT savings

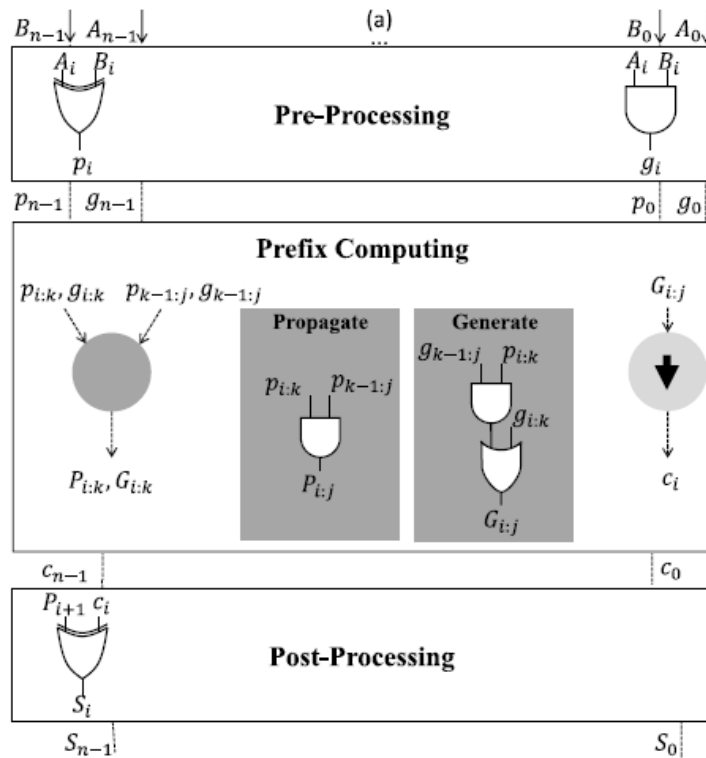


Limitations of Traditional PPAs

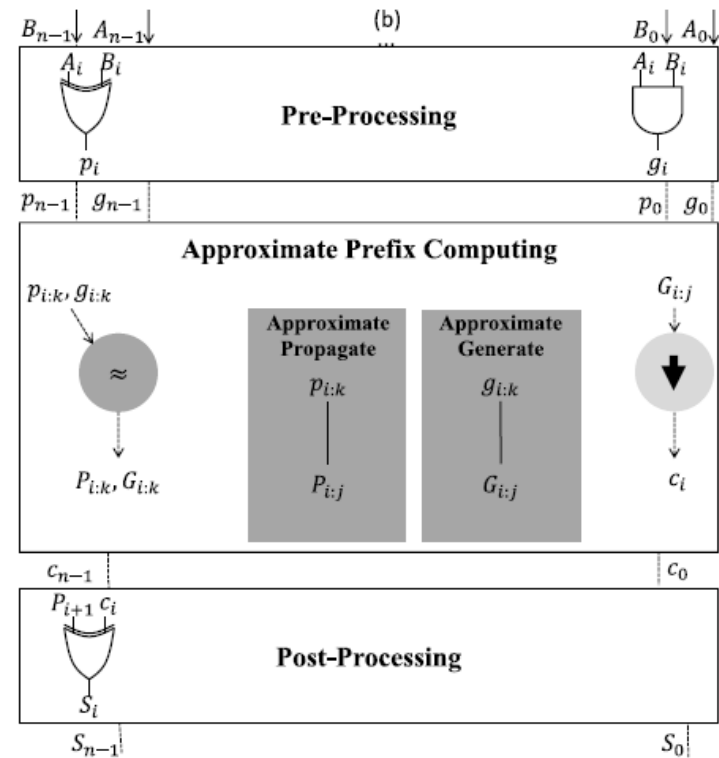
- ▶ All bits treated equally → wastes hardware for LSBs.
- ▶ LSB errors have minimal effect on total sum magnitude.
- ▶ Full accuracy everywhere is unnecessary for many applications.
- ▶ Need architecture with selective approximation.

AxPO: Approximate Prefix Operator

- Replaces exact prefix logic for selected bits.
- Reduces gate count in LSB region.



Our AxPPA proposal



AxPPA Architecture Overview

- ▶ Splits adder into:
 - ▶ Approximate region (K bits)
 - ▶ Exact prefix region ($W-K$ bits)
- ▶ Approximate part:
 - ▶ No carry propagation
 - ▶ Fast, small
- ▶ Exact part:
 - ▶ Builds prefix tree starting at bit K
 - ▶ Computes global carry correctly

Algorithm (AxPPA):

- Improves performance by avoiding unnecessary prefix calculations in LSB

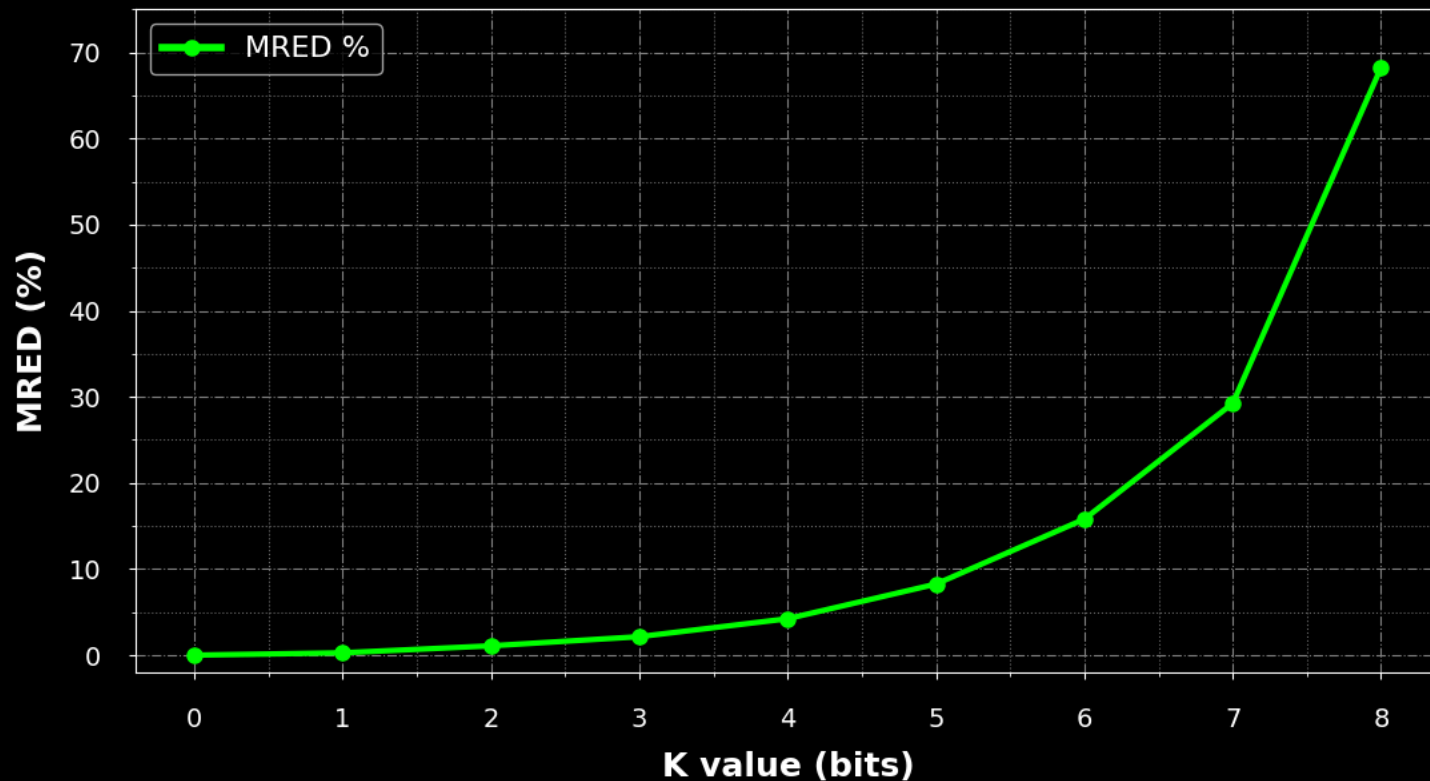
Algorithm 2 Pseudo Code for W -Bit Input and K -Bit AxPPA

```
1: Input:  $A, B, cin, W, K$ 
2: Output:  $S$ 
3: if  $K > 0$  then {Approximate part}
4:   for  $i = 0$  to  $K - 1$  do
5:      $p(1, i) \leftarrow A(i) \text{ XOR } B(i)$ 
6:      $g(1, i) \leftarrow A(i) \text{ AND } B(i)$ 
7:   end for
8:    $S(0) \leftarrow p(0)$ 
9:   for  $i = 1$  to  $K - 2$  do
10:     $S(1, i) \leftarrow p(1, i) \text{ XOR } g(1, i - 1)$ 
11:   end for
12:    $carry \leftarrow g(K - 1)$ 
13: end if
14: if  $(W - K) > 0$  then {Exact part}
15:   for  $i = K$  to  $W - 1$  do
16:      $p(1, i) \leftarrow A(i) \text{ XOR } B(i)$ 
17:      $g(1, i) \leftarrow A(i) \text{ AND } B(i)$ 
18:   end for
19:    $y \leftarrow 1$ 
20:   for  $i = (K + 1)$  to (Delay calculation in Table II) do
21:     for  $j = (K + y + 1)$  to  $W + 1$  do
22:        $P(i, j) \leftarrow p(i - 1, j) \text{ AND } p(i - 1, j - y)$ 
23:        $G(i, j) \leftarrow p(i - 1, j) \text{ AND } (g(i - 1, j - y) \text{ OR } g(i - 1, j))$ 
24:     end for
25:      $y \leftarrow 2^{j-1}$ 
26:   end for
27:    $cin \leftarrow carry$ 
28:   for  $i = (K + 1)$  to  $W + 1$  do
29:      $c(i) \leftarrow cin \text{ AND } P(W + 1, i) \text{ OR } G(W + 1, i)$ 
30:      $S(i) \leftarrow c(i - 1) \text{ XOR } P(W + 1, i)$ 
31:   end for
32: end if
```

Mean Relative Error Distance

$$\text{Mean Relative Error Distance} = \frac{1}{n} \sum_{i=1}^n \frac{|measured_i - actual_i|}{actual_i}.$$

MRED vs K for 8-bit AxPPA

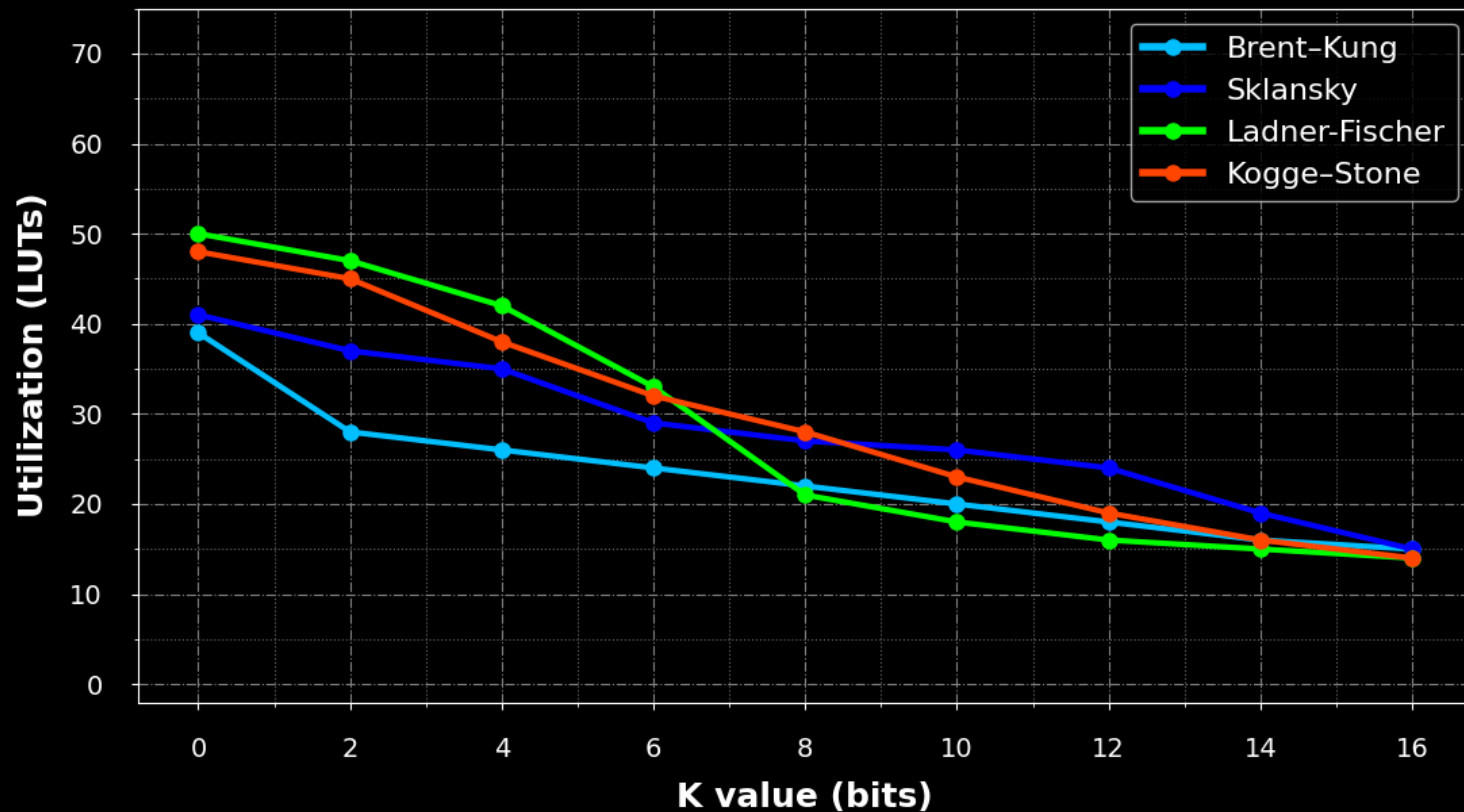


Observations from MRED

- ▶ MRED increases with K . More bits approximated $\rightarrow$ more LSB errors $\rightarrow$ accumulated error increases.
- ▶ Exact adders have zero MRED. No approximated carry or propagate logic $\rightarrow$ full PPA correctness.
- ▶ For small K (1-4), MRED rises extremely slowly. Only low-significance bits are approximated; MSBs remain exact $\rightarrow$ worst-case errors do not propagate upward heavily.
- ▶ For large K (12-16), MRED increases sharply. Losing carry chains in LSB region causes more deviations in intermediate sums $\rightarrow$ accumulated sum error is higher.

Utilization: Look Up Tables (LUTs)

Utilization vs K for 16-bit AxPPA

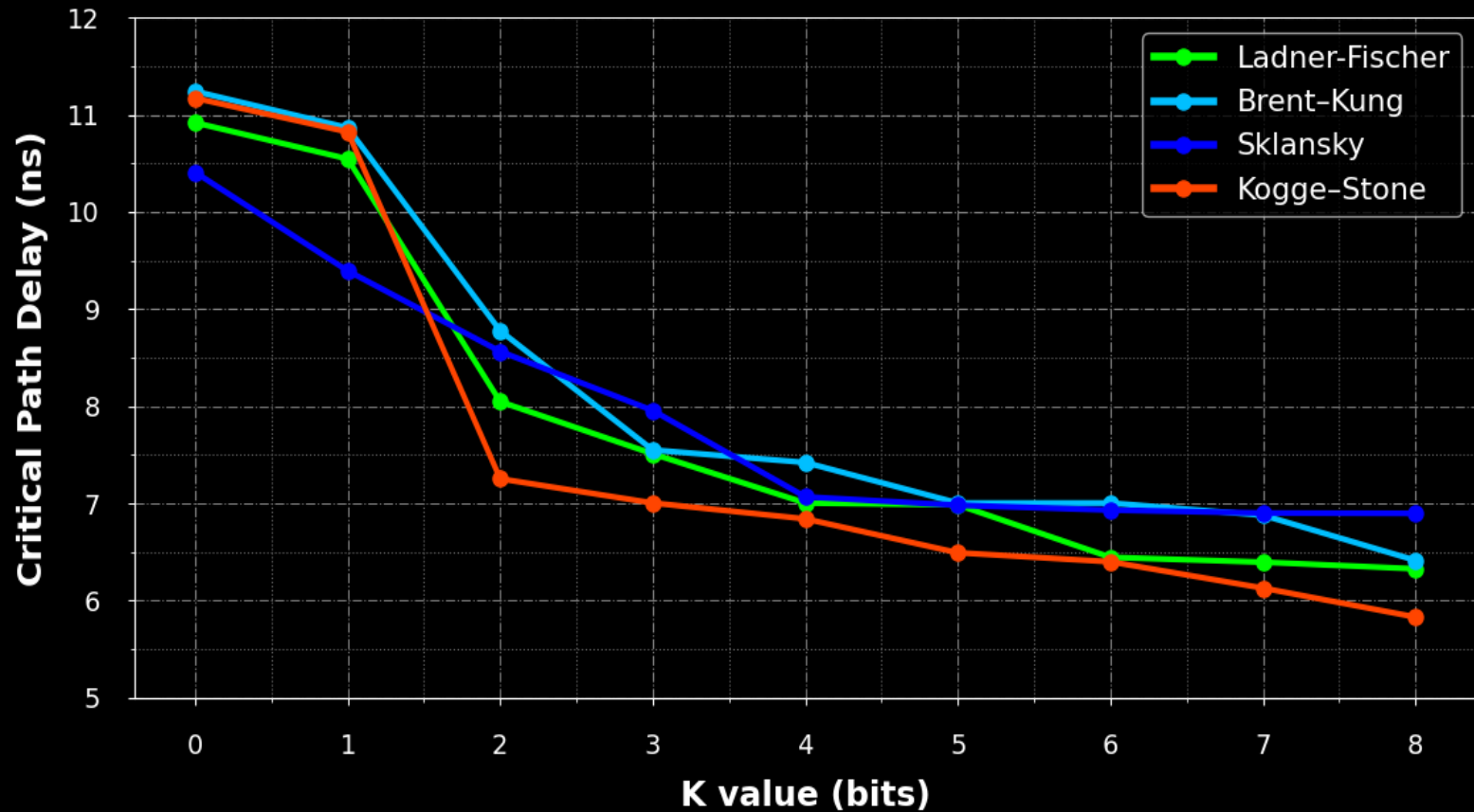


Observations from LUTs

- ▶ LUT count decreases almost linearly with K. Each increment in K removes a predictable number of POs in the prefix tree.
- ▶ Approximate PPA is significantly smaller than exact PPA. In exact PPAs, every bit needs propagate/generate logic; in AxPPA the LSB region uses only wires.
- ▶ Diminishing returns at high K. Once the LSB region becomes fully approximate, the remaining MSB region still uses full exact-prefix logic → area cannot reduce further.
- ▶ KS and LF give the highest LUT savings. KS has the largest number of prefix nodes → removing LSB nodes saves many LUTs. LF has a balanced prefix-tree size and generates largest savings per removed PO.
- ▶ BK shows the smallest LUT savings. BK has minimal POs to begin with → approximating K bits removes fewer logic nodes compared to KS/LF.
- ▶ SK gives moderate LUT reduction but less than KS/LF. High fan-out cells constrain logic removal & produce LUTs for routing and buffering.

Timing Analysis

Critical Path Delay vs K for 16-bit AxPPA

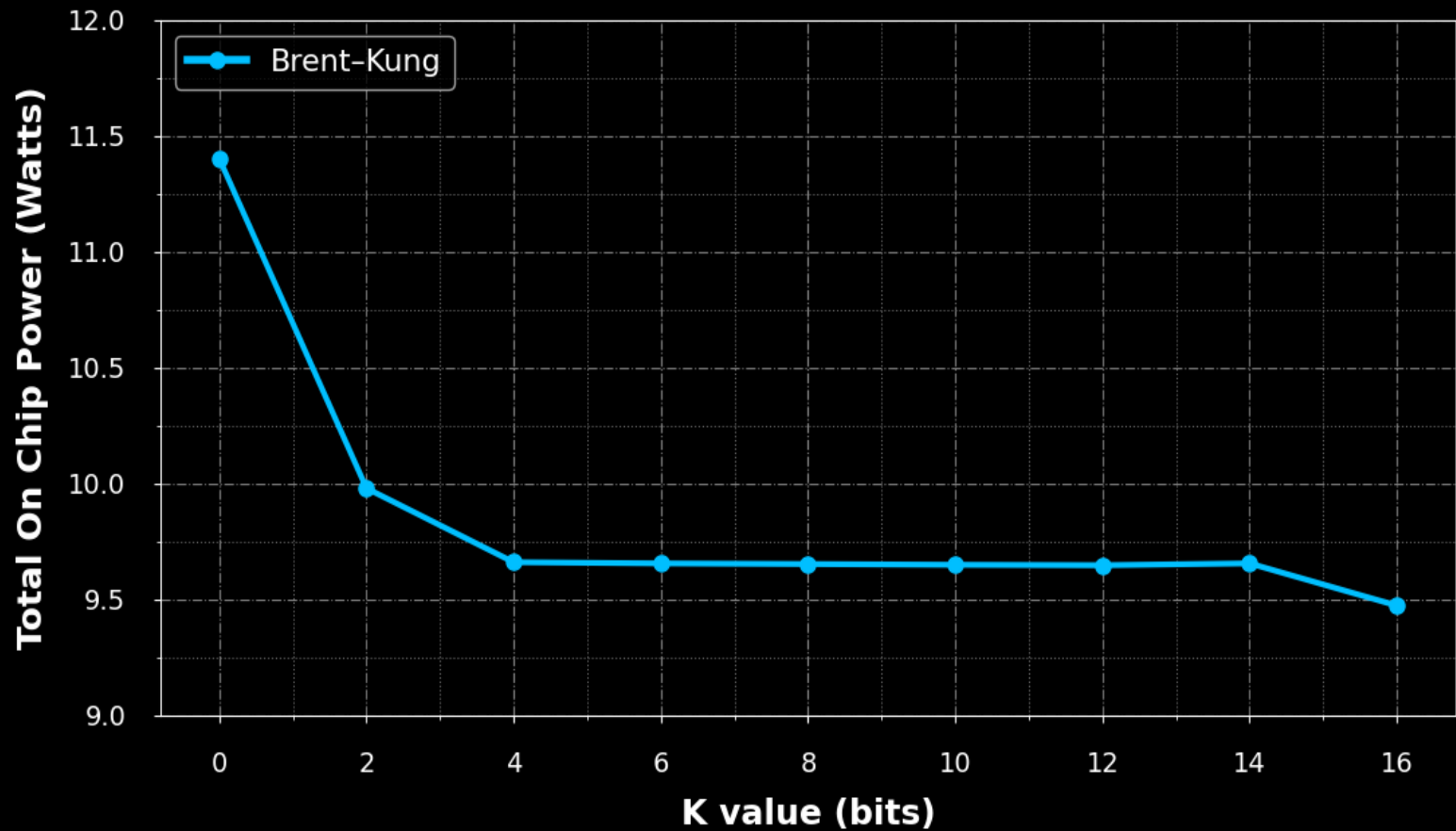


Observations from Timing Analysis

- ▶ Delay decreases as K increases. In approximate adders, the lower K bits do NOT go through the prefix tree (AxPO uses only wires). Removing LSB prefix logic shortens the critical path.
- ▶ Approximate adders always outperform exact adders in delay. Exact PPAs have full-depth prefix trees; approximate PPAs remove several levels of prefix computation in LSB region.
- ▶ Timing improvement becomes smaller at high K . Once most LSB logic is removed, the MSB exact-prefix logic determines delay. Further increasing K does not significantly shorten remaining critical paths.
- ▶ KS achieves the best delay reduction. KS is the deepest and most parallel prefix structure; removing its LSB POs drastically reduces intrinsic logic depth.
- ▶ LF has consistently good delay but not the best. LF has balanced prefix levels and lower fanout; delay improves well with K but initial overhead is smaller than KS, so absolute reduction is smaller.
- ▶ BK and SK show moderate timing gains. BK is Already optimized for low delay; removing LSB logic helps but not dramatically. SK has high fan-out stages; removing lower POs doesn't eliminate long connection paths in MSB region.

Power Analysis

Total On Chip Power vs K for 16-bit AxPPA



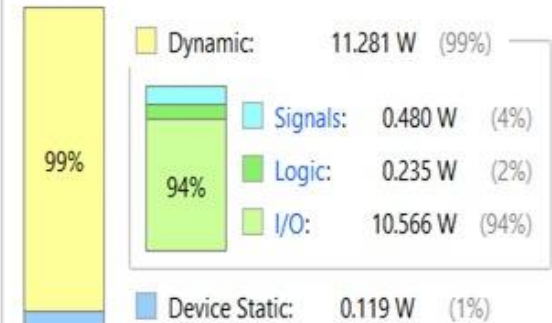
BK Power for K=0

Summary

Power estimation from Synthesized netlist. Activity derived from constraints files, simulation files or vectorless analysis. Note: these early estimates can change after implementation.

Total On-Chip Power:	11.401 W
Design Power Budget:	Not Specified
Power Budget Margin:	N/A
Junction Temperature:	82.0°C
Thermal Margin:	18.0°C (3.6 W)
Effective θ_{JA} :	5.0°C/W
Power supplied to off-chip devices:	0 W
Confidence level:	Low

On-Chip Power



BK Power for K=2

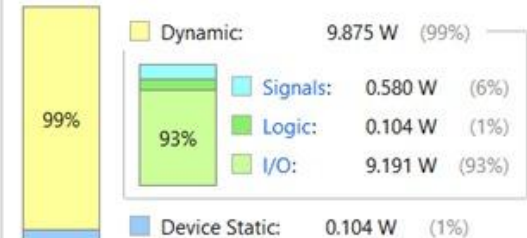
Summary

Power estimation from Synthesized netlist. Activity derived from constraints files, simulation files or vectorless analysis. Note: these early estimates can change after implementation.

Total On-Chip Power:	9.979 W
Design Power Budget:	Not Specified
Power Budget Margin:	N/A
Junction Temperature:	74.9°C
Thermal Margin:	25.1°C (5.0 W)
Effective θ_{JA} :	5.0°C/W
Power supplied to off-chip devices:	0 W
Confidence level:	Low

[Launch Power Constraint Advisor](#) to find and fix invalid switching activity

On-Chip Power



Observations from Power Analysis

- ▶ Approximate adders always consume less power than exact PPAs. Exact adders have full-width trees with many propagate/generate cells; approximate versions replace many of these with wires → near-zero switching.
- ▶ BK gives modest power savings. BK has fewer total logic elements → removing LSB logic has smaller absolute effect.

Summarizing Each Architecture

▶ Brent-Kung

- ▶ Most shallow tree structure (minimal POs)
- ▶ Smallest LUT savings
- ▶ Moderate delay reduction
- ▶ Least LUTs

▶ Kogge-Stone:

- ▶ Minimum delay.
- ▶ Largest number of prefix nodes
- ▶ High fanout
- ▶ Low per-node fan-out, high total node count
- ▶ Best LUT savings
- ▶ Best delay saving

▶ Sklanskey

- ▶ Computes wide prefixes aggressively in early stages.
- ▶ High per-node fan-out, low total node count
- ▶ Moderate LUT savings
- ▶ Moderate delay reduction

▶ Ladner-Fischer

- ▶ Most balanced prefix tree (neither too deep nor too shallow)
- ▶ Medium fanout
- ▶ Best LUT savings
- ▶ Consistently good delay

References

- ▶ <https://ieeexplore.ieee.org/document/9956923>
- ▶ <https://docs.amd.com/r/en-US/ug896-vivado-ip/Vivado-Design-Suite-Documentation>